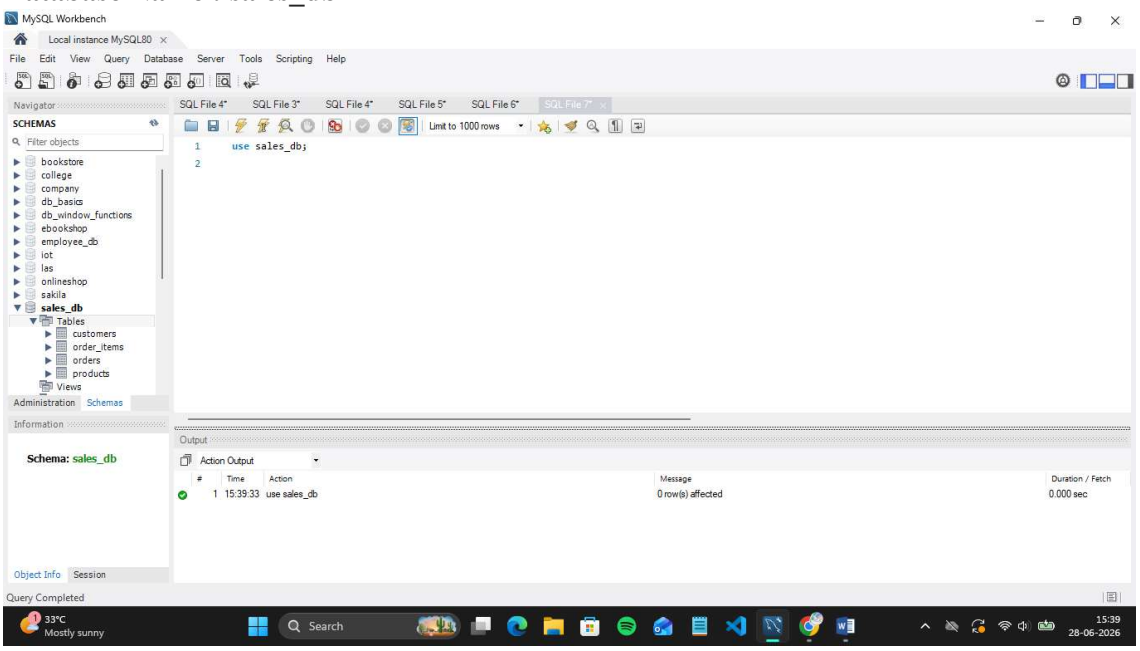
Week 2 Assignment

SQL-based data analysis using filtering, aggregation, and basic business queries

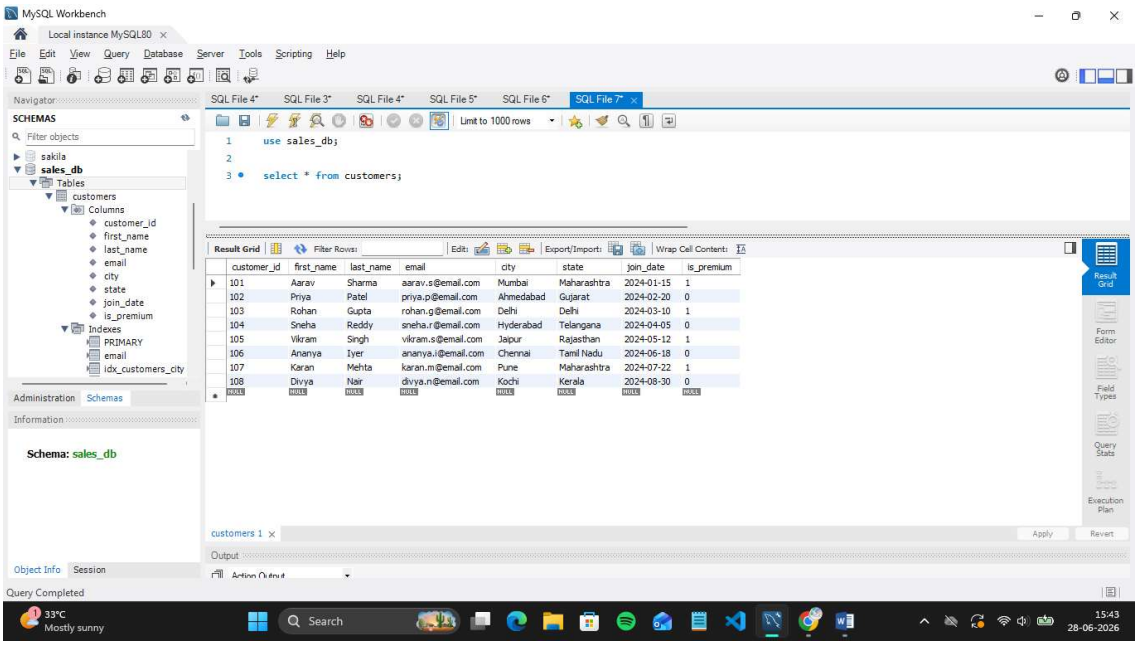
Name: Pathan Aman Firojkhan
Student ID- CT_CSI_DE_1157

Database :
Database Name : sales_db

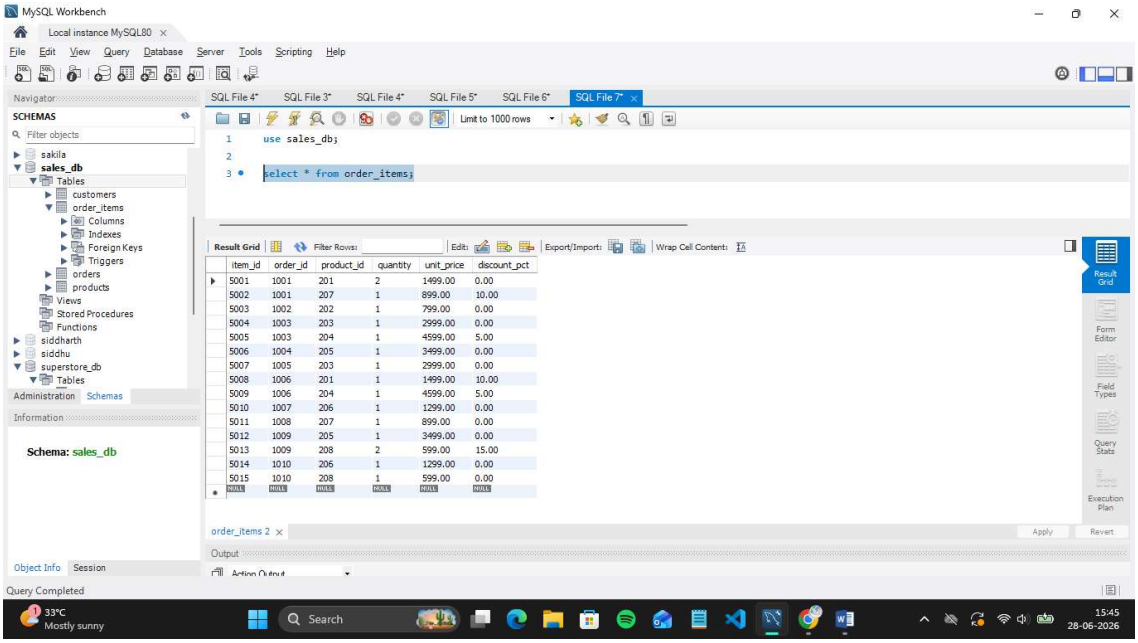


All Tables and Data of each Tables:

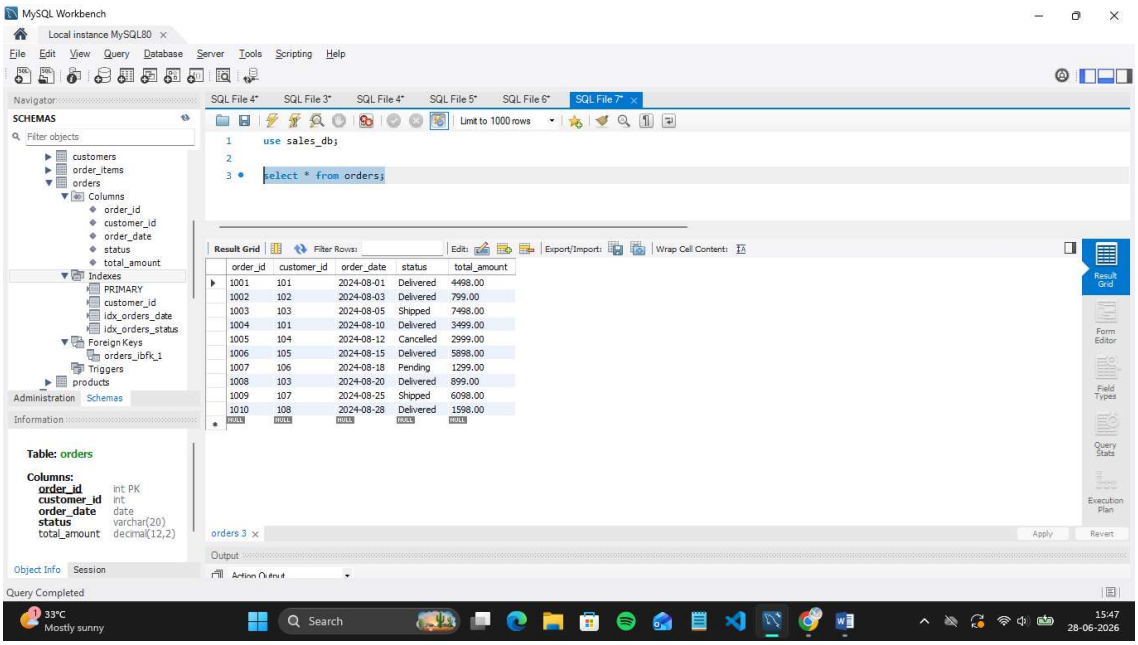
1. Table Name : customers



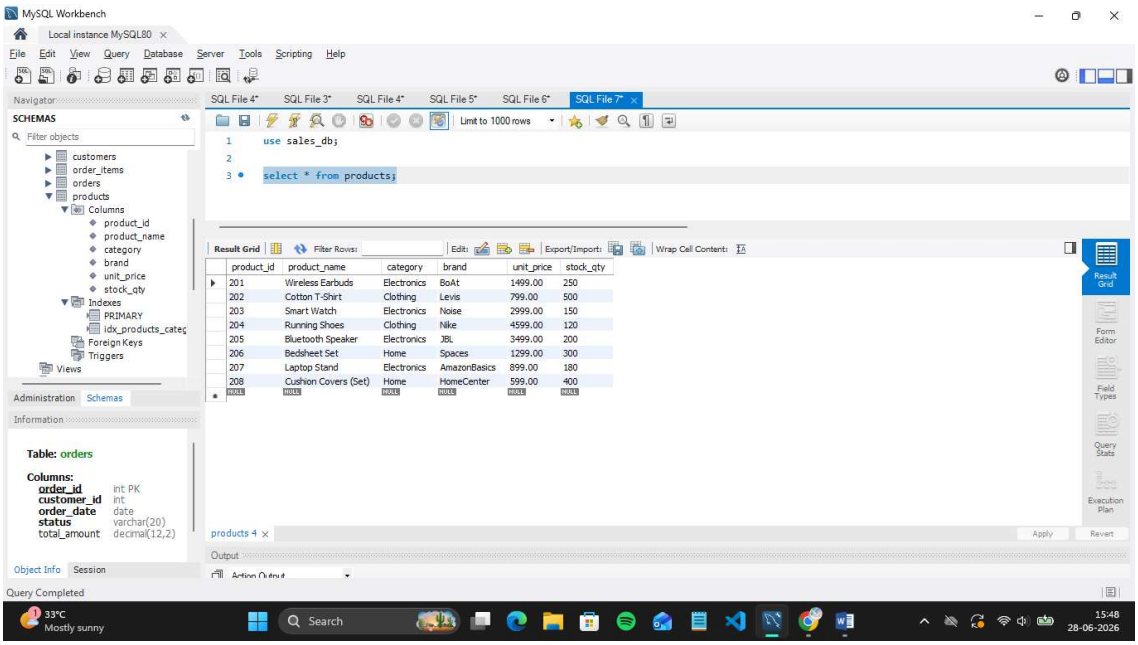
2. Table Name : order_items



3. Table Name : orders



4. Table Name : products



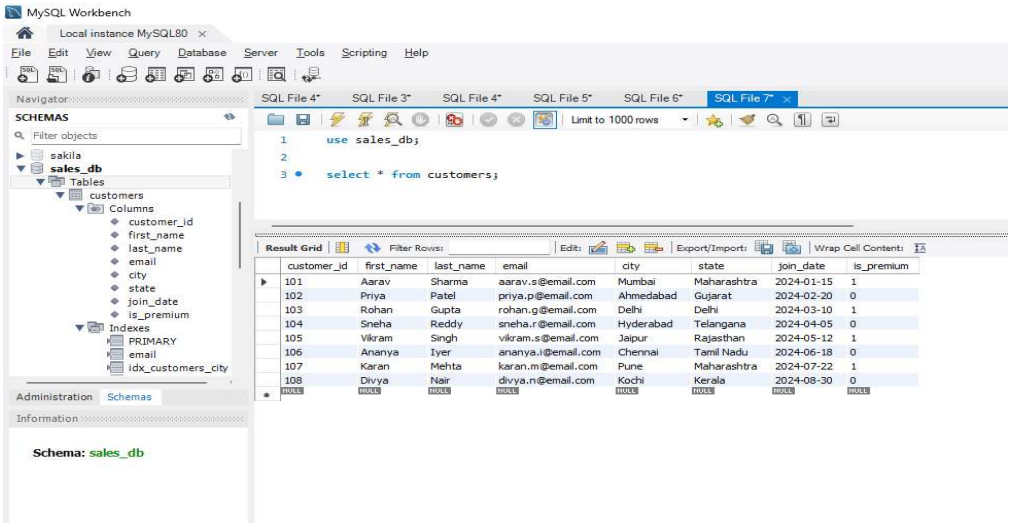
Section A — SQL Basics (SELECT, Constraints, Primary Keys)

Q1. Write a query to display all columns and rows from the customer's table.

SQL Query:

```
Select * from customers;
```

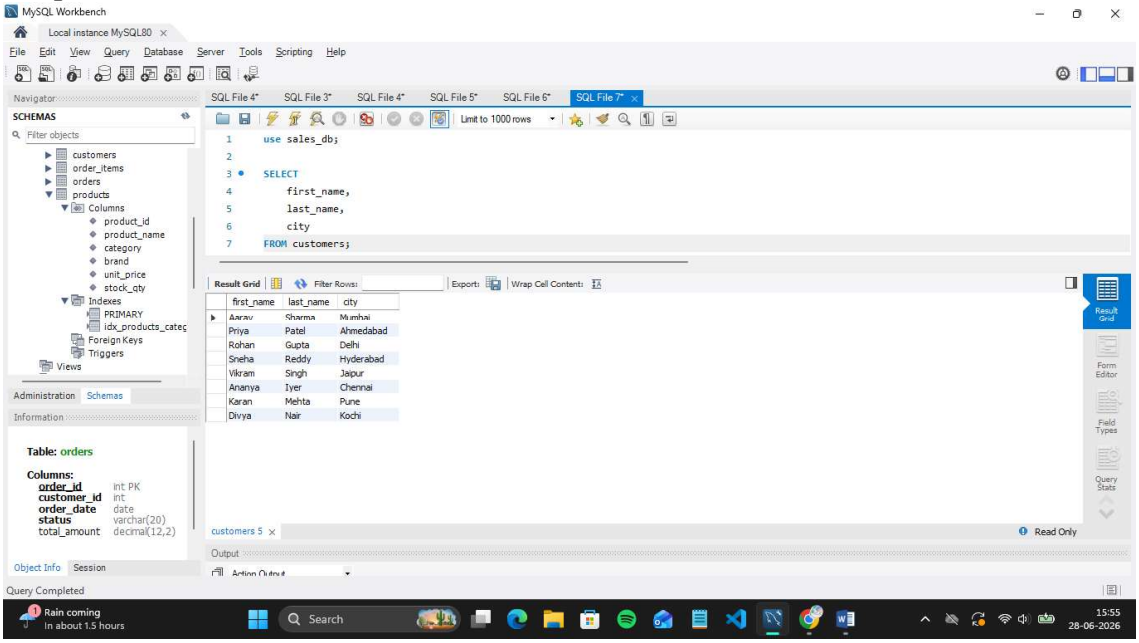
Output:



Q2. Retrieve only the first_name, last_name, and city of all customers.
SQL Query :

```
SELECT
    first_name,
    last_name,
    city
FROM customers;
```

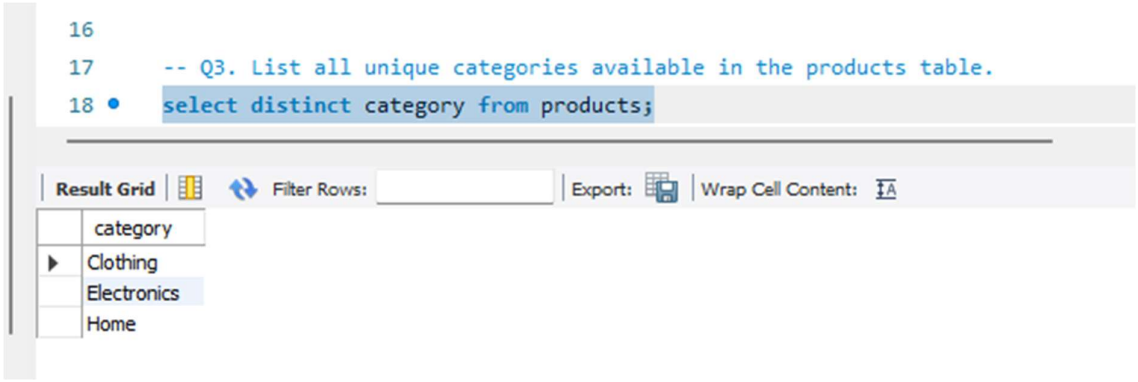
Output:



Q3. List all unique categories available in the products table.
SQL Query :

```
select distinct category from products;
```

Output :



Q4. Identify the Primary Key of each table in the schema. Explain why a Primary Key must be unique and NOT NULL.

SQL Query :

desc cutomers;

Output:

```
20
21      -- Q4. Identify the Primary Key of each table in the schema. Explain why a Primary Key must be unique and NOT NULL.
22 •    desc customers;
23
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
customer_id	int	NO	PRI	NULL	
first_name	varchar(50)	NO		NULL	
last_name	varchar(50)	NO		NULL	
email	varchar(100)	NO	UNI	NULL	
city	varchar(50)	NO	MUL	NULL	
state	varchar(50)	NO	MUL	NULL	
join_date	date	NO		NULL	
is_premium	tinyint(1)	YES		0	

SQL Query:

desc order_items;

Output :

```
20
21      -- Q4. Identify the Primary Key of each table in the schema. Explain why a Primary Key must be unique and NOT NULL.
22 •    desc customers;
23
24 •    desc order_items;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
item_id	int	NO	PRI	NULL	
order_id	int	NO	MUL	NULL	
product_id	int	NO	MUL	NULL	
quantity	int	NO		NULL	
unit_price	decimal(10,2)	NO		NULL	
discount_pct	decimal(5,2)	YES		0.00	

SQL Query:

desc orders;

Output:

```
25
26 •    desc orders;
```

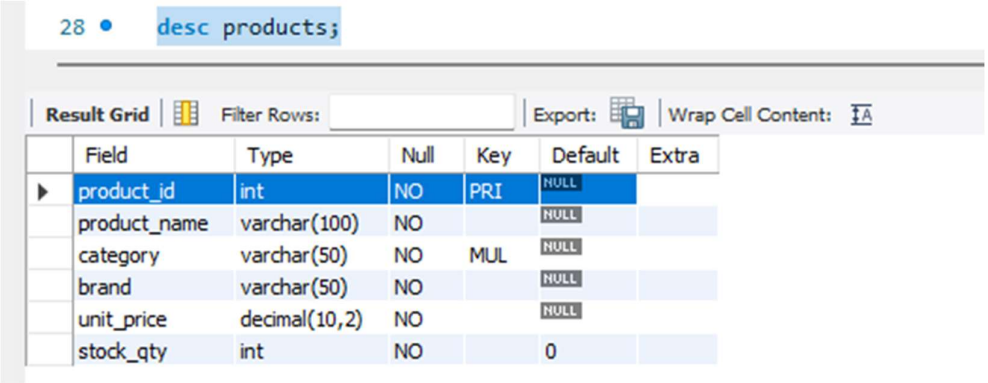
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
order_id	int	NO	PRI	NULL	
customer_id	int	NO	MUL	NULL	
order_date	date	NO	MUL	NULL	
status	varchar(20)	NO	MUL	Pending	
total_amount	decimal(12,2)	NO		NULL	

SQL Query :

desc products;

Output:



Field	Type	Null	Key	Default	Extra
product_id	int	NO	PRI	NULL	
product_name	varchar(100)	NO		NULL	
category	varchar(50)	NO	MUL	NULL	
brand	varchar(50)	NO		NULL	
unit_price	decimal(10,2)	NO		NULL	
stock_qty	int	NO		0	

Why must a Primary Key be UNIQUE and NOT NULL?

A Primary Key uniquely identifies every row in a table.

UNIQUE:

No duplicate values are allowed.

NOT NULL:

Every row must have an identifier.

Q5. What constraints are applied to the email column in the customers table? What would happen if you tried to insert a duplicate email?

This Schema will be used :
email VARCHAR(100) UNIQUE NOT NULL

Two constraints will be applied

1. UNIQUE Constraint

Means every customer must have a different email.

Example:

Allowed:

siddharth@email.com
siddhi@email.com

Not allowed:

siddhi@email.com
siddhi@email.com

2. NOT NULL Constraint

Means email cannot be empty.

Invalid:

INSERT INTO customers(email)
VALUES(NULL);

What happens if duplicate email is inserted?
ERROR: Duplicate entry 'aarav.s@email.com'
for key 'email'

```
32 •  select * from customers;
33 •  INSERT INTO customers
34    VALUES
35    (109,
36    'Rahul',
37    'Patil',
38    'aarav.s@email.com',
39    'Pune',
40    'Maharashtra',
41    '2024-09-01',
42    TRUE);
43
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
9	16:00:39	SELECT first_name, last_name, city FROM customers LIMIT 0, 1000	8 row(s) returned	0.000 sec / 0.000 sec
10	16:01:42	select distinct category from products LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
11	19:46:49	desc customers	8 row(s) returned	0.000 sec / 0.000 sec
12	19:49:23	desc order_items	6 row(s) returned	0.015 sec / 0.000 sec
13	19:51:31	desc orders	5 row(s) returned	0.000 sec / 0.000 sec
14	19:53:09	desc products	6 row(s) returned	0.000 sec / 0.000 sec
15	19:58:11	desc customers	8 row(s) returned	0.016 sec / 0.000 sec
16	20:01:29	select * from customers LIMIT 0, 1000	8 row(s) returned	0.000 sec / 0.000 sec
17	20:02:14	INSERT INTO customers VALUES (109, 'Rahul', 'Patil', 'aarav.s@email.com', 'Pune', 'Ma...	Error Code: 1062. Duplicate entry 'aarav.s@email.com' for key 'customers.email'	0.031 sec

Q6. Try inserting a product with unit_price = -50. What happens and which constraint prevents it? Write both the INSERT statement and explain the error.

The product table has:

unit_price DECIMAL(10,2)
CHECK(unit_price > 0)

```sql
-- Q6. Try inserting a product with unit_price = -50. What happens and which constraint prevents it? Write both the INSERT statement and explain t

INSERT INTO products
VALUES
(
209,
'Gaming Mouse',
'Electronics',
'Logitech',
-50,
100
);
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
11	19:46:49	desc customers	8 row(s) returned	0.000 sec / 0.000 sec
12	19:49:23	desc order_items	6 row(s) returned	0.015 sec / 0.000 sec
13	19:51:31	desc orders	5 row(s) returned	0.000 sec / 0.000 sec
14	19:53:09	desc products	6 row(s) returned	0.000 sec / 0.000 sec
15	19:58:11	desc customers	8 row(s) returned	0.016 sec / 0.000 sec
16	20:01:29	select * from customers LIMIT 0, 1000	8 row(s) returned	0.000 sec / 0.000 sec
17	20:02:14	INSERT INTO customers VALUES (109, 'Rahul', 'Patil', 'aarav.s@email.com', 'Pune', 'Ma...	Error Code: 1062. Duplicate entry 'aarav.s@email.com' for key 'customers.email'	0.031 sec
18	20:06:30	INSERT INTO products VALUES (209, 'Gaming Mouse', 'Electronics', 'Logitech', -50, 100)	Error Code: 3819. Check constraint 'products_chk_1' is violated.	0.015 sec

The database rejects this query.

Error example:

CHECK constraint failed:

unit_price > 0

Explanation:

The CHECK constraint validates data before inserting.

Condition:

unit_price > 0

Means:

Allowed:

499

Not allowed:

0

-50

-100

Because negative product prices are logically invalid.

Section B — Filtering & Optimization (WHERE, Indexes)

Q7. Retrieve all orders with status = 'Delivered'.

SQL Query :

```
SELECT *
FROM orders
WHERE status = 'Delivered';
```

Output:

```
58    -- Section B – Filtering & Optimization (WHERE, Indexes)
59
60    -- Q7. Retrieve all orders with status = 'Delivered'.
61    SELECT *
62    FROM orders
63    WHERE status = 'Delivered';
64
65
66
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

order_id	customer_id	order_date	status	total_amount
1001	101	2024-08-01	Delivered	4498.00
1002	102	2024-08-03	Delivered	799.00
1004	101	2024-08-10	Delivered	3499.00
1006	105	2024-08-15	Delivered	5898.00
1008	103	2024-08-20	Delivered	899.00
1010	108	2024-08-28	Delivered	1598.00
NULL	NULL	NULL	NULL	NULL

Q8. Find all products in the 'Electronics' category with a unit_price greater than ₹2000.

SQL Query :

```
SELECT *
FROM products
WHERE category = 'Electronics'
AND unit_price > 2000;
```

Output :

```
64
65    -- Q8. Find all products in the 'Electronics' category with a unit_price greater than ₹2000.
66
67 •  SELECT *
68    FROM products
69    WHERE category = 'Electronics'
70    AND unit_price > 2000;
71
72
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

product_id	product_name	category	brand	unit_price	stock_qty
203	Smart Watch	Electronics	Noise	2999.00	150
205	Bluetooth Speaker	Electronics	JBL	3499.00	200
NULL	NULL	NULL	NULL	NULL	NULL

Q9. List all customers who joined in the year 2024 and belong to the state 'Maharashtra'.

SQL Query :

```
SELECT *
FROM customers
WHERE state = 'Maharashtra'
AND YEAR(join_date) = 2024;
```

Output:

```
72    -- Q9. List all customers who joined in the year 2024 and belong to the state 'Maharashtra'.
73
74 •  SELECT *
75    FROM customers
76    WHERE state = 'Maharashtra'
77    AND YEAR(join_date) = 2024;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q10. Find all orders placed between '2024-08-10' and '2024-08-25' (inclusive) that are NOT cancelled.

SQL Query:

```
SELECT *
FROM orders
WHERE order_date BETWEEN '2024-08-10'
AND '2024-08-25'
AND status <> 'Cancelled';
```

Output:

```
79    -- Q10. Find all orders placed between '2024-08-10' and '2024-08-25' (inclusive) that are NOT cancelled.
80
81 •  SELECT *
82    FROM orders
83    WHERE order_date BETWEEN '2024-08-10'
84    AND '2024-08-25'
85    AND status <> 'Cancelled';
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

order_id	customer_id	order_date	status	total_amount
1004	101	2024-08-10	Delivered	3499.00
1006	105	2024-08-15	Delivered	5898.00
1007	106	2024-08-18	Pending	1299.00
1008	103	2024-08-20	Delivered	899.00
1009	107	2024-08-25	Shipped	6098.00
NULL	NULL	NULL	NULL	NULL

Q11. Explain what the index idx_orders_date does. How would it improve the performance of a query that filters orders by order_date? Write a sample query that would benefit from this index.

The schema contains:

CREATE INDEX idx_orders_date
ON orders(order_date);

What is an Index?
An index is a database structure that improves the speed of searching data.
It works similar to an index page in a book.

Without an index:
Database checks every row:
Order 1
Order 2
Order 3
...
Order 100000
This is called:
Full Table Scan

With an index:
Database directly finds matching dates.
Example:
2024-08-01 → Order 1001
2024-08-05 → Order 1003
2024-08-10 → Order 1004

Query benefiting from this index:

```
SELECT *
FROM orders
WHERE order_date = '2024-08-10';
```

The database can quickly locate orders on that date using:
idx_orders_date

Benefits:

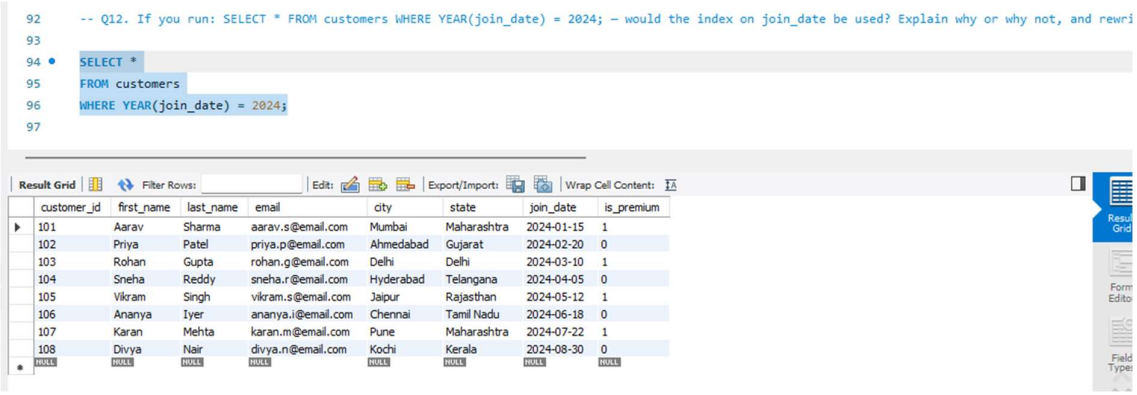
- Faster filtering
- Faster sorting by date
- Better query performance for large tables

Q12. If you run: SELECT * FROM customers WHERE YEAR(join_date) = 2024; — would the index on join_date be used? Explain why or why not, and rewrite the query to be index-friendly (SARGable).

SQL Query :

```
SELECT *
FROM customers
WHERE YEAR(join_date) = 2024;
```

Output :



```
92    -- Q12. If you run: SELECT * FROM customers WHERE YEAR(join_date) = 2024; — would the index on join_date be used? Explain why or why not, and rewri
93
94 •  SELECT *
95    FROM customers
96    WHERE YEAR(join_date) = 2024;
97
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Answer:

Usually **NO**, the index will not be used efficiently.

Why?

Because we are applying a function:

YEAR(join_date)

The database must first calculate the year for every row.

Example:

Before searching:

2024-01-15 → YEAR() → 2024
2023-05-10 → YEAR() → 2023
2025-02-20 → YEAR() → 2025

Because the column is modified, the index cannot directly search.

This query is **not SARGable**.

Index-friendly (SARGable) Query:

```
SELECT *
FROM customers
WHERE join_date >= '2024-01-01'
AND join_date < '2025-01-01';
```

Why is this better?

The database can directly search the date range:

2024-01-01
|
|
2024-12-31

The index can quickly locate matching rows.

SARGable vs Non-SARGable

Type	Example	Index Usage
SARGable	join_date >= '2024-01-01'	Uses index
Non-SARGable	YEAR(join_date)=2024	Usually avoids index

Section C — Aggregation (GROUP BY, SUM, COUNT, AVG, MIN, MAX)

Q13. Count the total number of orders in the orders table.

SQL Query:

Select count(*) as total_orders from orders;

Output:

```
99      -- Section C — Aggregation (GROUP BY, SUM, COUNT, AVG, MIN, MAX)
100     -- Q13. Count the total number of orders in the orders table.
101
102 •   SELECT COUNT(*) AS total_orders
103     FROM orders;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

total_orders
10

Q14. Find the total revenue (SUM of total_amount) from all 'Delivered' orders.

SQL Query :

```
SELECT
 SUM(total_amount) AS total_revenue
FROM orders
WHERE status = 'Delivered';
```

Output:

```
105     -- Q14. Find the total revenue (SUM of total_amount) from all 'Delivered' orders.
106
107 •   SELECT
108         SUM(total_amount) AS total_revenue
109     FROM orders
110     WHERE status = 'Delivered';
111
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

total_revenue
17191.00

Q15. Calculate the average unit_price of products in each category.

SQL Query :

```
SELECT
```

```
category,
AVG(unit_price) AS average_price
FROM products
GROUP BY category;
```

Output:

```
112     -- Q15. Calculate the average unit_price of products in each category.
113
114 •   SELECT
115         category,
116         AVG(unit_price) AS average_price
117     FROM products
118     GROUP BY category;
119
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	average_price
Clothing	2699.000000
Electronics	2224.000000
Home	949.000000

Q16. For each order status, find the count of orders and the total revenue. Sort the result by total revenue in descending order.

SQL Query:

```
SELECT
   status,
   COUNT(order_id) AS order_count,
   SUM(total_amount) AS total_revenue
FROM orders
GROUP BY status
ORDER BY total_revenue DESC;
```

Output:

```
120     -- Q16. For each order status, find the count of orders and the total revenue. Sort the result by total revenue in descending order.
121
122 •   SELECT
123         status,
124         COUNT(order_id) AS order_count,
125         SUM(total_amount) AS total_revenue
126     FROM orders
127     GROUP BY status
128     ORDER BY total_revenue DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

status	order_count	total_revenue
Delivered	6	17191.00
Shipped	2	13596.00
Cancelled	1	2999.00
Pending	1	1299.00

Q17. Find the most expensive (MAX) and cheapest (MIN) product in each category.
SQL Query:

```
SELECT
   category,
   MAX(unit_price) AS highest_price,
   MIN(unit_price) AS lowest_price
FROM products
GROUP BY category;
```

Output:

```
130     -- Q17. Find the most expensive (MAX) and cheapest (MIN) product in each category.
131
132 •   SELECT
133         category,
134         MAX(unit_price) AS highest_price,
135         MIN(unit_price) AS lowest_price
136     FROM products
137     GROUP BY category;
138
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	highest_price	lowest_price
Clothing	4599.00	799.00
Electronics	3499.00	899.00
Home	1299.00	599.00

599.00

Q18. List all product categories where the average unit_price is greater than ₹2000. (Hint: Use HAVING clause)

SQL Query:

```
SELECT
   category,
   AVG(unit_price) AS average_price
FROM products
GROUP BY category
HAVING AVG(unit_price) > 2000;
```

Output:

```
139     -- Q18. List all product categories where the average unit_price is greater than ₹2000. (Hint: Use HAVING clause)
140
141 •   SELECT
142         category,
143         AVG(unit_price) AS average_price
144     FROM products
145     GROUP BY category
146     HAVING AVG(unit_price) > 2000;
147
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	average_price
Clothing	2699.000000
Electronics	2224.000000

Section D — Joins & Relationships

Q19. Write an INNER JOIN query to display each order along with the customer's first_name and last_name.
Show: order_id, order_date, first_name, last_name, total_amount.

SQL Query:

```
SELECT
    orders.order_id,
    orders.order_date,
    customers.first_name,
    customers.last_name,
    orders.total_amount
FROM orders
INNER JOIN customers
ON orders.customer_id = customers.customer_id;
```

Output:

```
150    -- Q19. Write an INNER JOIN query to display each order along with the customer's first_name and last_name. Show: order_id, order_date, first_name,
151    SELECT
152        orders.order_id,
153        orders.order_date,
154        customers.first_name,
155        customers.last_name,
156        orders.total_amount
157    FROM orders
158    INNER JOIN customers
159    ON orders.customer_id = customers.customer_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_id	order_date	first_name	last_name	total_amount
1001	2024-08-01	Aarav	Sharma	4498.00
1004	2024-08-10	Aarav	Sharma	3499.00
1002	2024-08-03	Priya	Patel	799.00
1003	2024-08-05	Rohan	Gupta	7498.00
1008	2024-08-20	Rohan	Gupta	899.00
1005	2024-08-12	Sneha	Reddy	2999.00
1006	2024-08-15	Vikram	Singh	5898.00
1007	2024-08-18	Ananya	Iyer	1299.00
1009	2024-08-25	Karan	Mehta	6098.00
1010	2024-08-28	Divya	Nair	1598.00

Result 27

Q20. Using a LEFT JOIN, list ALL customers and their orders (if any). Customers with no orders should still appear with NULL values for order columns.

SQL Query:

```
SELECT
    customers.customer_id,
    customers.first_name,
    customers.last_name,
    orders.order_id,
    orders.order_date,
    orders.total_amount
FROM customers
LEFT JOIN orders
ON customers.customer_id = orders.customer_id;
```

Output:

```
163 •  SELECT
164        customers.customer_id,
165        customers.first_name,
166        customers.last_name,
167        orders.order_id,
168        orders.order_date,
169        orders.total_amount
170    FROM customers
171    LEFT JOIN orders
172    ON customers.customer_id = orders.customer_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	first_name	last_name	order_id	order_date	total_amount
101	Aarav	Sharma	1001	2024-08-01	4498.00
101	Aarav	Sharma	1004	2024-08-10	3499.00
102	Priya	Patel	1002	2024-08-03	799.00
103	Rohan	Gupta	1003	2024-08-05	7498.00
103	Rohan	Gupta	1008	2024-08-20	899.00
104	Sneha	Reddy	1005	2024-08-12	2999.00
105	Vikram	Singh	1006	2024-08-15	5898.00
106	Ananya	Iyer	1007	2024-08-18	1299.00
107	Karan	Mehta	1009	2024-08-25	6098.00
108	Divya	Nair	1010	2024-08-28	1598.00

Result 28

Q21. Write a query using JOINs across three tables (orders → order_items → products) to show: order_id, product_name, quantity, unit_price, and discount_pct for each order item.

SQL Query:

```
SELECT
   orders.order_id,
   products.product_name,
   order_items.quantity,
   order_items.unit_price,
   order_items.discount_pct
FROM orders

INNER JOIN order_items
ON orders.order_id = order_items.order_id

INNER JOIN products
ON order_items.product_id = products.product_id;
```

Output:

```sql
SELECT
    orders.order_id,
    products.product_name,
    order_items.quantity,
    order_items.unit_price,
    order_items.discount_pct
FROM orders
INNER JOIN order_items
ON orders.order_id = order_items.order_id
INNER JOIN products
ON order_items.product_id = products.product_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

rder_id	product_name	quantity	unit_price	discount_pct
)01	Wireless Earbuds	2	1499.00	0.00
)06	Wireless Earbuds	1	1499.00	10.00
)02	Cotton T-Shirt	1	799.00	0.00
)03	Smart Watch	1	2999.00	0.00
)05	Smart Watch	1	2999.00	0.00
)03	Running Shoes	1	4599.00	5.00
)06	Running Shoes	1	4599.00	5.00
)04	Bluetooth Speaker	1	3499.00	0.00

: 29

Q22. Explain the difference between LEFT JOIN and RIGHT JOIN with an example from this schema. When would you use a FULL OUTER JOIN?

SQL Query:

SELECT *
FROM customers
LEFT JOIN orders
ON customers.customer_id = orders.customer_id;

Output:

```sql
-- Q22. Explain the difference between LEFT JOIN and RIGHT JOIN with an example from this schema. When would you use a FULL OUTER JOIN?

SELECT *
FROM customers
LEFT JOIN orders
ON customers.customer_id = orders.customer_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	first_name	last_name	email	city	state	join_date	is_premium	order_id	customer_id	order_date	status	total_amount
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1	1001	101	2024-08-01	Delivered	4498.00
101	Aarav	Sharma	aarav.s@e	aarav.s@email.com	Maharashtra	2024-01-15	1	1004	101	2024-08-10	Delivered	3499.00
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0	1002	102	2024-08-03	Delivered	799.00
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1	1003	103	2024-08-05	Shipped	7498.00
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1	1008	103	2024-08-20	Delivered	899.00
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0	1005	104	2024-08-12	Cancelled	2999.00
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1	1006	105	2024-08-15	Delivered	5898.00
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0	1007	106	2024-08-18	Pending	1299.00

Result 30

Q23. Identify all Foreign Key relationships in the schema. Explain what would happen if you tried to insert an order with customer_id = 999 (which doesn't exist in customers).

Foreign Key 1:
orders table
customer_id
References:
customers(customer_id)
Relationship:
customers
  |
  ↓
orders

Foreign Key 2:
order_items table
order_id
References:
orders(order_id)
Relationship:
orders
  |
  ↓
order_items

Foreign Key 3:
order_items table
product_id
References:
products(product_id)
Relationship:
products
  |
  ↓
order_items

What happens if we insert order with customer_id = 999?
Query:

```
INSERT INTO orders
VALUES
(
1011,
999,
'2024-09-01',
'Pending',
1598
);
```

Result:

The database rejects the insertion.
Error:
Foreign Key Constraint Failed

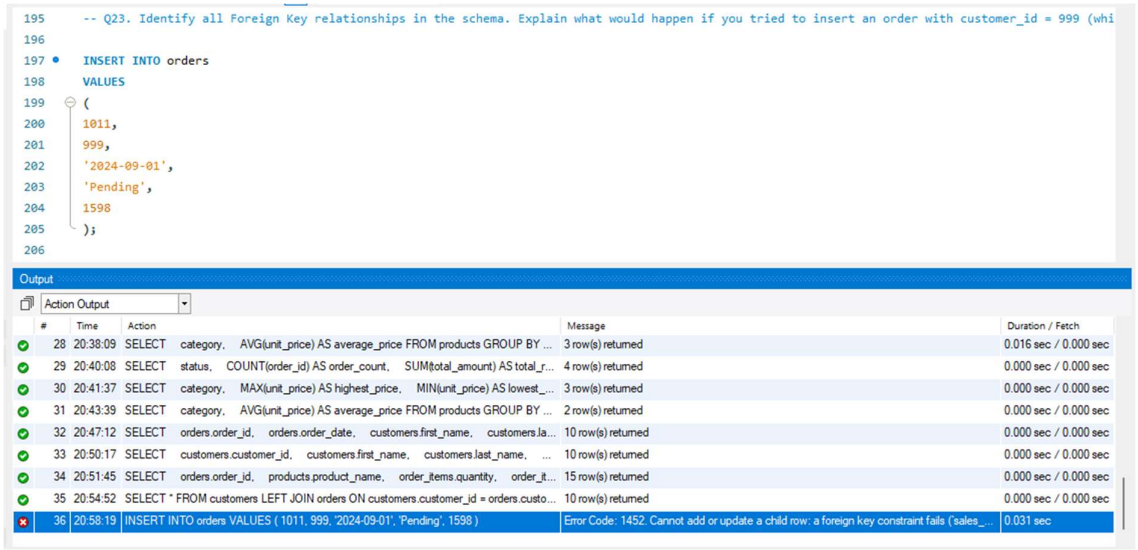
Why?
Because:
customer_id = 999
does not exist in:
customers table
The foreign key prevents invalid references.

Why are Foreign Keys Important?
They maintain:
1. Data Integrity
Example:
Cannot create an order for a non-existing customer.

2. Relationship Accuracy
Every order must belong to a valid customer.

3. Prevent Orphan Records
Invalid data like:
Order 1011 → Customer 999
is prevented.

Output:



Section E — Advanced Concepts (CASE, ACID, Transactions)

Q24. Write a query using CASE to classify products into price tiers:
- **'Budget' → unit_price < 1000**
- **'Mid-Range' → unit_price BETWEEN 1000 AND 3000**
- **'Premium' → unit_price > 3000**

Display: product_name, unit_price, price_tier.

SQL Query:

```
SELECT
   product_name,
   unit_price,
   CASE
      WHEN unit_price < 1000 THEN 'Budget'
      WHEN unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range'
      WHEN unit_price > 3000 THEN 'Premium'
   END AS price_tier
FROM products;
```

Output:

```
214
215 •  SELECT
216        product_name,
217        unit_price,
218        CASE
219            WHEN unit_price < 1000 THEN 'Budget'
220            WHEN unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range'
221            WHEN unit_price > 3000 THEN 'Premium'
222        END AS price_tier
223    FROM products;
224
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_name	unit_price	price_tier
Wireless Earbuds	1499.00	Mid-Range
Cotton T-Shirt	799.00	Budget
Smart Watch	2999.00	Mid-Range
Running Shoes	4599.00	Premium
Bluetooth Speaker	3499.00	Premium
Bedsheet Set	1299.00	Mid-Range
Laptop Stand	899.00	Budget
Cushion Covers (Set)	599.00	Budget

Q25. Using a CASE statement inside an aggregate function, count how many orders are 'Delivered' vs 'Not Delivered' (all other statuses). Display the result in a single row.

SQL Query:

```
SELECT
   COUNT(
      CASE
         WHEN status = 'Delivered'
         THEN 1
```

```
    END
  ) AS delivered_orders,

  COUNT(
    CASE
      WHEN status <> 'Delivered'
      THEN 1
    END
  ) AS not_delivered_orders

FROM orders;
```

Output:

```
227 • SELECT
228     COUNT(
229         CASE
230             WHEN status = 'Delivered'
231             THEN 1
232         END
233     ) AS delivered_orders,
234     COUNT(
235         CASE
236             WHEN status <> 'Delivered'
237             THEN 1
238         END
239     ) AS not_delivered_orders
240 FROM orders;
241
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

delivered_orders	not_delivered_orders
6	4

Q26. Explain each letter of ACID:

- **A – Atomicity**
- **C – Consistency**
- **I – Isolation**
- **D – Durability**

Give a real-world example (e.g., bank transfer) showing why each property is important.

ACID properties ensure database transactions are reliable and safe.

ACID stands for:

Letter	Meaning
A	Atomicity
C	Consistency
I	Isolation
D	Durability

A — Atomicity
Meaning:
A transaction is treated as a single unit.

Either: All operations succeed

OR: All operations fail

Example: Bank Transfer

Transfer ₹500 from Account A to Account B.

Steps:

1. Deduct ₹500 from A
2. Add ₹500 to B

If step 1 succeeds but step 2 fails:

The database cancels the entire transaction.

Money is not lost.

C — Consistency
Meaning:
Database must always remain valid before and after a transaction.

Rules and constraints must be followed.

Example:

Before transfer:

Account A = ₹5000
Account B = ₹2000
After transfer:

Account A = ₹4500
Account B = ₹2500
Total money remains:

₹7000

I — Isolation
Meaning:
Multiple transactions happening together should not affect each other incorrectly.

Example:

Two users trying to buy the last available product.

Transaction 1:

Buy product
Transaction 2:

Buy same product
Isolation ensures the database handles them correctly.

D — Durability
Meaning:
Once a transaction is committed, the changes permanently exist.

Example:

After successful payment:

Payment Successful
Even if the server crashes:

The payment record remains saved.

ACID Summary:

Property	Purpose
Atomicity	Complete or cancel transaction
Consistency	Maintain valid data
Isolation	Prevent transaction conflicts
Durability	Save committed changes permanently

Q27. Write a SQL transaction that does the following atomically:
1. Insert a new order (order_id=1011, customer_id=102, today's date, 'Pending', 1598.00)
2. Insert two order items for that order
3. Update the stock_qty of the purchased products
4. If any step fails, ROLLBACK the entire transaction. Otherwise, COMMIT.
Write the complete BEGIN...COMMIT/ROLLBACK block.

SQL Query:

```
START TRANSACTION;


-- Step 1: Insert new order

INSERT INTO orders
(
   order_id,
   customer_id,
   order_date,
   status,
   total_amount
)
VALUES
(
   1011,
   102,
   CURRENT_DATE,
   'Pending',
   1598.00
);



-- Step 2: Insert order items

INSERT INTO order_items
(
   item_id,
   order_id,
   product_id,
   quantity,
   unit_price,
   discount_pct
)
VALUES
(
```

```
  5016,
  1011,
  206,
  1,
  1299.00,
  0
);



INSERT INTO order_items
(
   item_id,
   order_id,
   product_id,
   quantity,
   unit_price,
   discount_pct
)
VALUES
(
5017,
1011,
208,
1,
599.00,
0
);



-- Step 3: Update stock quantity

UPDATE products
SET stock_qty = stock_qty - 1
WHERE product_id = 206;


UPDATE products
SET stock_qty = stock_qty - 1
WHERE product_id = 208;



-- Step 4: Commit changes
```

COMMIT;

Output:

```
321  );
322
323
324
325  -- Step 3: Update stock quantity
326
327  UPDATE products
328  SET stock_qty = stock_qty - 1
329  WHERE product_id = 206;
330
331
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
32	20:47:12	SELECT orders.order_id, orders.order_date, customers.first_name, customers.la...	10 row(s) returned	0.000 sec / 0.000 sec
33	20:50:17	SELECT customers.customer_id, customers.first_name, customers.last_name, ...	10 row(s) returned	0.000 sec / 0.000 sec
34	20:51:45	SELECT orders.order_id, products.product_name, order_items.quantity, order_it...	15 row(s) returned	0.000 sec / 0.000 sec
35	20:54:52	SELECT * FROM customers LEFT JOIN orders ON customers.customer_id = orders.custo...	10 row(s) returned	0.000 sec / 0.000 sec
36	20:58:19	INSERT INTO orders VALUES (1011, 999, '2024-09-01', 'Pending', 1598)	Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails (`sales_...	0.031 sec
37	21:02:02	INSERT INTO orders VALUES (1011, 999, '2024-09-01', 'Pending', 1598)	Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails (`sales_...	0.031 sec
38	21:02:27	SELECT product_name, unit_price, CASE WHEN unit_price < 1000 THEN '...	8 row(s) returned	0.000 sec / 0.000 sec
39	21:04:39	SELECT product_name, unit_price, CASE WHEN unit_price < 1000 THEN '...	8 row(s) returned	0.000 sec / 0.000 sec
40	21:34:06	SELECT COUNT(CASE WHEN status = 'Delivered' THEN 1 ...	1 row(s) returned	0.000 sec / 0.000 sec
41	21:38:03	COMMIT	0 row(s) affected	0.015 sec